

Instituto Politécnico Nacional
"La Técnica al Servicio de la Patria"



Escuela Superior de Cómputo

IMPLEMENTACIÓN DE ALGORITMO PSO

Práctica 2

Alcántara López Wendy Yuritzi
Robles Guzmán Naomi Isabel

6AV1

Profesor: Ary Shared Rosas Carrillo

Bioinformática
24 de septiembre de 2026

Índice

1 Introducción **1**

2 Marco Teórico **2**
2.1 Introducción a la Computación Evolutiva y Optimización por Enjambres . . . 2
2.2 Optimización por Enjambre de Partículas (PSO) . . . 2
2.3 Formulación matemática del algoritmo . . . 3
2.4 Parámetros del algoritmo . . . 4
2.5 El dilema exploración vs. explotación . . . 5
2.6 PSO con inercia dinámica . . . 6

3 Desarrollo de la práctica **7**
3.1 Variante 1: PSO con parámetros fijos (clásico) . . . 7
3.1.1 Interfaz común para las estrategias de w . . . 7
3.2 Variante 2: PSO con inercia dinámica . . . 7
3.2.1 Inercia lineal decreciente . . . 7
3.2.2 Inercia basada en el fitness . . . 8
3.2.3 Integración en el algoritmo . . . 9

4 Resultados **11**
4.1 Error final por configuración . . . 11
4.2 Distribución del error . . . 12
4.3 Sensibilidad al valor de w y escalabilidad . . . 12

5 Resultados: coeficientes fijos frente a dinámicos **14**
5.1 Mejor solución, media y desviación estándar . . . 14
5.2 Convergencia . . . 15
5.3 Pruebas estadísticas . . . 16
5.4 Comparación con los resultados de la Sección 4 . . . 16

6 Conclusiones **18**

Bibliografía **19**

1 Introducción

Los problemas de optimización aparecen en prácticamente todas las áreas de la ingeniería y la ciencia, desde el ajuste de parámetros en modelos de aprendizaje automático hasta el diseño de estructuras y el control de sistemas dinámicos. Muchos de estos problemas no tienen una solución analítica sencilla: la función objetivo puede ser no lineal, no convexa, o carecer de una expresión derivable que permita aplicar métodos clásicos basados en gradiente. En estos casos, las metaheurísticas bioinspiradas ofrecen una alternativa práctica, ya que buscan soluciones aceptables sin requerir información detallada sobre la estructura del problema.

La Optimización por Enjambre de Partículas (PSO, por sus siglas en inglés) es una de estas metaheurísticas. Fue propuesta por Kennedy y Eberhart en 1995 a partir de la observación del comportamiento colectivo de bandadas de aves y bancos de peces, donde cada individuo ajusta su trayectoria con base en su propia experiencia y en la información compartida por el grupo. Esta idea se tradujo en un algoritmo donde un conjunto de partículas explora el espacio de búsqueda, actualizando su posición y velocidad en función de su mejor resultado histórico y del mejor resultado encontrado por el enjambre.

El comportamiento del algoritmo depende en gran medida de sus parámetros de control, en particular del peso de inercia, que regula el equilibrio entre exploración y explotación durante la búsqueda. En su formulación original, este peso permanece constante a lo largo de todas las iteraciones. Sin embargo, trabajos posteriores mostraron que variar la inercia de forma dinámica, ya sea disminuyéndola o aumentándola conforme avanza el proceso, puede mejorar la capacidad del algoritmo para converger hacia mejores soluciones.

En esta práctica se implementa primero la versión clásica de PSO, con parámetros fijos, y después se extiende en dos pasos. Primero se hace variable únicamente el peso de inercia w, con una estrategia lineal decreciente y otra basada en el fitness de cada partícula. Después se hacen variables los tres coeficientes w, c_1 y c_2, siguiendo el esquema de coeficientes de aceleración variables en el tiempo (TVAC). El objetivo es comparar estos enfoques y analizar el efecto que tiene cada modificación sobre el comportamiento del enjambre y la calidad de las soluciones obtenidas.

2 Marco Teórico

2.1 Introducción a la Computación Evolutiva y Optimización por Enjambres

La Computación Evolutiva agrupa un conjunto de técnicas metaheurísticas de optimización inspiradas en procesos naturales, dentro de las cuales se ubican los algoritmos bio-inspirados de inteligencia colectiva o *swarm intelligence*. A diferencia de los algoritmos evolutivos clásicos (como los algoritmos genéticos), que basan su búsqueda en operadores de selección, cruza y mutación aplicados sobre una población de individuos, los algoritmos de enjambre prescinden de dichos operadores genéticos y en su lugar hacen evolucionar a la población mediante la cooperación y la competencia entre sus propios integrantes [1].

Esta familia de algoritmos toma como referencia el comportamiento colectivo observado en sistemas biológicos, como las bandadas de aves, los bancos de peces o las colonias de insectos, en los cuales ningún individuo posee información global del entorno, pero el intercambio de información local entre vecinos permite que el grupo en su conjunto localice recursos o soluciones de manera eficiente. El sociobiólogo E. O. Wilson planteó ya en 1975 que, en teoría, los miembros de un banco de peces pueden beneficiarse de los descubrimientos y la experiencia previa de todos los demás miembros durante la búsqueda de alimento, idea que sirvió de base conceptual para trasladar este principio de "información compartida" al ámbito de la optimización computacional [2].

Dentro de la inteligencia de enjambres se distinguen principalmente dos vertientes: la Optimización por Colonia de Hormigas (*Ant Colony Optimization*) y la Optimización por Enjambre de Partículas (*Particle Swarm Optimization*, PSO), siendo esta última el algoritmo objeto de este trabajo. Ambas comparten el principio de que una población de agentes simples, que interactúan siguiendo reglas locales, puede dar lugar a un comportamiento colectivo capaz de resolver problemas de optimización complejos que serían difíciles de abordar mediante métodos puramente individuales.

2.2 Optimización por Enjambre de Partículas (PSO)

La Optimización por Enjambre de Partículas fue propuesta originalmente por James Kennedy y Russell Eberhart en 1995, como un método para la optimización de funciones no lineales continuas. El algoritmo surgió a partir de la simulación de un modelo social simplificado, por lo que, aunque su formulación conserva la metáfora social que le dio origen, el algoritmo puede sostenerse y aplicarse de manera independiente de dicha metáfora [3].

En su planteamiento inicial, Kennedy y Eberhart se apoyaron en los trabajos previos de Craig Reynolds sobre modelos de bandadas (*flocking*) y observaron que, al simplificar dicho modelo eliminando las reglas relacionadas con la evasión de colisiones entre individuos, el comportamiento visual del grupo dejaba de asemejarse a una bandada y pasaba a comportarse como un enjambre; de ahí el nombre de *Particle Swarm Optimization* [2].

En PSO, la población recibe el nombre de enjambre (*swarm*) y cada individuo se denomina partícula. Cada partícula representa una posible solución candidata dentro de un espacio de búsqueda de n dimensiones y se desplaza por dicho espa-

cio ajustando su trayectoria en función de dos fuentes de información: su propia experiencia (la mejor posición que ella misma ha encontrado hasta el momento) y la experiencia del enjambre en su conjunto (la mejor posición encontrada por cualquier partícula del grupo). Ninguna partícula desaparece durante la ejecución del algoritmo; en cambio, el enjambre completo converge progresivamente hacia regiones prometedoras del espacio de búsqueda mediante un mecanismo de intercambio de información entre sus miembros [4].

PSO se clasifica como un algoritmo metaheurístico, ya que realiza pocas o ninguna suposición sobre el problema que se está optimizando, lo que le permite explorar espacios de soluciones candidatas de gran tamaño y dimensionalidad. Entre sus principales ventajas se encuentran la sencillez conceptual de su implementación —que puede programarse en pocas líneas de código—, el bajo costo computacional en términos de memoria y velocidad, y el requerir únicamente operadores matemáticos primitivos [3].

A partir de su formulación original, el algoritmo ha dado lugar a numerosas variantes orientadas a mejorar su desempeño y a resolver problemas de convergencia identificados en versiones posteriores de estudio, entre las que destaca la incorporación del peso de inercia (*inertia weight*) propuesto por Shi y Eberhart, así como el factor de constricción introducido por Clerc y Kennedy en 2002, ambos orientados a equilibrar la exploración y la explotación del espacio de búsqueda a lo largo de las iteraciones [5].

2.3 Formulación matemática del algoritmo

En PSO, cada partícula i del enjambre se representa mediante un vector de posición X_i y un vector de velocidad V_i dentro de un espacio de búsqueda de n dimensiones. En cada iteración, la posición de la partícula se actualiza sumando su velocidad actual a su posición actual [6]:

$$\vec{X}_{id}(t+1) = \vec{X}_{id}(t) + \vec{V}_{id}(t+1) \tag{1}$$

donde el subíndice d denota la dimensión considerada. La velocidad, a su vez, se actualiza de acuerdo con la siguiente expresión:

$$\vec{V}_{id}(t+1) = w\,\vec{V}_{id}(t) + c_1 r_1 \left(P_{id} - \vec{X}_{id}(t)\right) + c_2 r_2 \left(G_d - \vec{X}_{id}(t)\right) \tag{2}$$

Los elementos de esta ecuación tienen el siguiente significado:

- w**:** peso de inercia (*inertia weight*), parámetro incorporado posteriormente a la formulación original por Shi y Eberhart para controlar la influencia de la velocidad previa de la partícula sobre su velocidad actual, regulando así el balance entre exploración y explotación del espacio de búsqueda.

- P_{id}**:** mejor posición histórica encontrada por la propia partícula i en la dimensión d (componente cognitivo o de "nostalgia simple", en términos de los autores originales).

- G_d**:** mejor posición encontrada por cualquier partícula del enjambre completo en la dimensión d (componente social).

- c_1, c_2**:** coeficientes de aceleración cognitivo y social, respectivamente, que ponderan la atracción de la partícula hacia su propio mejor histórico y hacia el mejor global del enjambre. Kennedy y Eberhart asignaron originalmente un valor de 2 a ambos coeficientes, de modo que el producto con un número aleatorio de media 0.5 diera, en promedio, un peso de 1, otorgando a la partícula probabilidades equivalentes de sobrepasar o quedarse corta respecto de su objetivo.

- r_1, r_2**:** números aleatorios distribuidos uniformemente en el intervalo $[0, 1]$, que introducen un componente estocástico en la búsqueda e impiden que todas las partículas converjan exactamente por la misma trayectoria.

Respecto al equilibrio entre los componentes cognitivo y social, los estudios experimentales de los autores originales mostraron que un valor de c_1 considerablemente mayor que c_2 provoca que las partículas exploren de forma errática y aislada el espacio de búsqueda, mientras que la situación inversa —un c_2 mucho mayor que c_1— conduce a una convergencia prematura hacia óptimos locales; los mejores resultados se obtienen generalmente cuando ambos coeficientes toman valores aproximadamente iguales [2].

El algoritmo se ejecuta de manera iterativa: en cada ciclo se evalúa la función objetivo (o función de aptitud) para la posición actual de cada partícula, se actualizan P_{id} y G_d cuando corresponde, y se recalculan velocidad y posición según las ecuaciones anteriores, repitiendo el proceso hasta alcanzar un criterio de paro, ya sea un número máximo de iteraciones o un umbral de convergencia sobre la calidad de la solución encontrada.

2.4 Parámetros del algoritmo

El desempeño de PSO depende en gran medida de la elección de sus parámetros de control. A diferencia de otros métodos de optimización, el algoritmo tiene pocos parámetros, pero pequeños cambios en su valor pueden hacer que el enjambre converja rápidamente, se estanque en un óptimo local o incluso diverja [7, 8].

- **Tamaño del enjambre (N):** número de partículas que exploran el espacio de búsqueda. Un enjambre grande cubre más regiones del espacio en cada iteración, pero aumenta el número de evaluaciones de la función objetivo. En la práctica se usan típicamente entre 20 y 50 partículas, y el algoritmo es poco sensible a este valor dentro de ese rango [7, 9].

- **Número de iteraciones (T):** determina el presupuesto de cómputo y funciona como criterio de paro. En las estrategias de inercia dinámica también define la velocidad con que cambian los coeficientes, ya que estos se expresan en función de la razón t/T.

- **Peso de inercia (w):** escala la velocidad de la iteración anterior. Shi y Eberhart [10] lo introdujeron para controlar el alcance de la búsqueda y encontraron que los valores entre 0.9 y 1.2 daban buenos resultados con $c_1 = c_2 = 2$. Si $w > 1$ la velocidad tiende a crecer en cada iteración y el enjambre puede divergir, mientras que un valor cercano a cero elimina la memoria del movimiento y la partícula depende solo de las atracciones hacia P_i y G [8].

- **Coeficientes de aceleración (c_1, c_2):** ponderan la atracción cognitiva y social. Su valor clásico es $c_1 = c_2 = 2$ [11], aunque se ha demostrado que su efecto no puede analizarse por separado del de w, porque los tres coeficientes determinan juntos si la trayectoria de la partícula converge u oscila [12].

- **Velocidad máxima (V_{max}):** límite que se aplica a cada componente de la velocidad para evitar que las partículas salgan del espacio de búsqueda. En la versión original era el único mecanismo para controlar la explosión de las velocidades. Con la incorporación de w su papel se volvió secundario, y suele fijarse como una fracción del rango de cada variable [13].

Clerc y Kennedy [12] analizaron la estabilidad de la trayectoria de una partícula y propusieron el factor de constricción χ, que multiplica toda la ecuación de velocidad:

$$\chi = \frac{2}{\left| 2 - \varphi - \sqrt{\varphi^2 - 4\varphi} \right|}, \qquad \varphi = c_1 + c_2 > 4. \tag{3}$$

Con el valor habitual $\varphi = 4.1$ se obtiene $\chi \approx 0.7298$. Esta formulación es algebraicamente equivalente a usar un peso de inercia $w = 0.7298$ y coeficientes $c_1 = c_2 = 0.7298 \cdot 2.05 \approx 1.49618$. Por esta razón estos valores se han vuelto una configuración de referencia en la literatura [7, 13], y son los que se usan en la implementación con coeficientes fijos de esta práctica.

2.5 El dilema exploración vs. explotación

Todo algoritmo de búsqueda metaheurística debe equilibrar dos objetivos opuestos. La *exploración* consiste en visitar regiones nuevas del espacio de búsqueda para no pasar por alto zonas prometedoras. La *explotación* consiste en refinar la búsqueda alrededor de las mejores soluciones encontradas para mejorar su precisión [14]. Un algoritmo que explora demasiado se comporta como una búsqueda aleatoria y tarda en converger. Uno que explota demasiado converge rápido, pero puede quedar atrapado en el primer óptimo local que encuentre, un fenómeno conocido como convergencia prematura [14, 15].

En PSO, cada término de la ecuación de velocidad contribuye de forma distinta a este equilibrio:

- El término de inercia $w\,V_i$ mantiene la dirección de movimiento de la partícula. Un valor grande de w le permite recorrer distancias largas y favorece la exploración global, mientras que un valor pequeño concentra la búsqueda en una región local [8, 10].

- El componente cognitivo $c_1 r_1 (P_i - X_i)$ atrae a cada partícula hacia su propia mejor posición. Como cada partícula tiene un P_i distinto, este término mantiene la diversidad del enjambre y favorece la exploración [16].

- El componente social $c_2 r_2 (G - X_i)$ atrae a todas las partículas hacia el mismo punto. Este término acelera la convergencia y favorece la explotación, pero también reduce la diversidad del enjambre [16].

El equilibrio adecuado no es el mismo durante toda la ejecución. Al inicio conviene explorar, porque el enjambre todavía no tiene información sobre la ubicación del óptimo. Al final conviene explotar, porque el enjambre ya identificó una región prometedora y lo que importa es refinar la solución [9, 15]. Esto es especialmente importante en funciones multimodales como Rastrigin, cuya gran cantidad de óptimos locales castiga tanto la convergencia prematura como la falta de convergencia. Un conjunto de coeficientes fijos representa un único compromiso entre ambos objetivos para toda la ejecución, lo que motiva el uso de coeficientes que cambian con el tiempo.

2.6 PSO con inercia dinámica

En la variante con inercia dinámica, w deja de ser constante y se ajusta durante la ejecución para pasar de un comportamiento exploratorio a uno explotador. Bansal et al. [17] compararon quince estrategias propuestas en la literatura, que pueden agruparse en tres familias:

- **Estrategias dependientes del tiempo.** w cambia en función de la iteración, sin usar información del enjambre. La más conocida es la inercia lineal decreciente (LDIW, por sus siglas en inglés) de Shi y Eberhart [9]:

$$w(t) = w_{max} - (w_{max} - w_{min})\,\frac{t}{T}, \tag{4}$$

  con los valores recomendados $w_{max} = 0.9$ y $w_{min} = 0.4$. Existen también versiones no lineales, en las que w decrece siguiendo una potencia de t/T [18].

- **Estrategias aleatorias.** w se muestrea en cada iteración, por ejemplo $w = 0.5 + \text{rand}()/2$, lo que produce un valor medio de 0.75. Eberhart y Shi [19] la propusieron para problemas cuyo óptimo cambia con el tiempo, donde no tiene sentido reducir w de forma monótona.

- **Estrategias adaptativas.** w se calcula a partir del estado del enjambre, por ejemplo la calidad de la solución de cada partícula, la tasa de partículas que mejoraron su mejor posición o la dispersión del enjambre [15, 20]. A diferencia de las anteriores, estas estrategias pueden asignar un valor de w distinto a cada partícula. La estrategia basada en fitness de esta práctica pertenece a esta familia.

La misma idea puede extenderse a los coeficientes de aceleración. Ratnaweera et al. [16] propusieron los coeficientes de aceleración variables en el tiempo (TVAC), en los que c_1 disminuye y c_2 aumenta linealmente durante la ejecución:

$$c_1(t) = c_{1i} + (c_{1f} - c_{1i})\,\frac{t}{T}, \qquad c_2(t) = c_{2i} + (c_{2f} - c_{2i})\,\frac{t}{T}. \tag{5}$$

Con $c_{1i} = 2.5$, $c_{1f} = 0.5$, $c_{2i} = 0.5$ y $c_{2f} = 2.5$, al inicio domina el componente cognitivo y las partículas se mueven de forma independiente. Al final domina el componente social y el enjambre se concentra alrededor del mejor global. Combinado con la inercia de la ecuación (4), este esquema obtuvo mejores resultados que el PSO con inercia lineal y coeficientes fijos en la mayoría de las funciones de prueba evaluadas por sus autores [16].

3 Desarrollo de la práctica

3.1 Variante 1: PSO con parámetros fijos (clásico)

En la variante clásica el coeficiente de inercia w tiene el mismo valor durante toda la ejecución. Este coeficiente determina qué tanto conserva cada partícula su velocidad anterior en la actualización

$$v_i^{t+1} = w\, v_i^t + c_1 r_1 \left(p_i - x_i^t\right) + c_2 r_2 \left(g - x_i^t\right), \tag{6}$$

donde p_i es la mejor posición encontrada por la partícula i y g es la mejor posición del enjambre. Un w alto favorece la exploración porque las partículas recorren distancias grandes sin cambiar mucho de dirección. Un w bajo favorece la explotación porque el movimiento depende casi por completo de las atracciones hacia p_i y g.

3.1.1 Interfaz común para las estrategias de w

Para comparar las estrategias sin duplicar el algoritmo, cada una se implementó como una función que recibe la iteración actual `t`, el número total de iteraciones `T`, el vector con el fitness de las partículas y el generador de números aleatorios `rng`. La función devuelve un escalar cuando todas las partículas comparten el mismo w, o un vector con un valor por partícula. Cada estrategia se construye con una función externa que fija sus parámetros y devuelve la función interna `w`. Con esta forma, el w constante de la variante 1 también se puede expresar como una estrategia:

Código 1: Estrategia de w constante (variante 1)

```
def w_constante(valor):
    # Enfoque 1: w fijo durante todas las iteraciones.
    def w(t, T, fitness, rng):
        return valor
    return w
```

3.2 Variante 2: PSO con inercia dinámica

En la inercia dinámica, w cambia durante la ejecución para ajustar ese equilibrio. En esta práctica se implementaron dos estrategias: una que reduce w linealmente con las iteraciones y otra que asigna a cada partícula un w calculado a partir de su fitness.

3.2.1 Inercia lineal decreciente

La primera estrategia empieza con un valor alto de w y lo reduce en cada iteración hasta un valor mínimo:

$$w(t) = w_{max} - \frac{(w_{max} - w_{min})\, t}{T}, \tag{7}$$

donde t es la iteración actual y T el número total de iteraciones. Se usaron $w_{max} = 0.9$ y $w_{min} = 0.4$, los mismos valores del cuaderno original. Al inicio las partículas se mueven con mucha inercia y exploran el espacio de búsqueda; conforme avanza la

ejecución, la inercia disminuye y el enjambre se concentra alrededor de las mejores soluciones encontradas. Como el ciclo recorre $t = 0, 1, \ldots, T-1$, el último valor aplicado es $0.9 - 0.5 \cdot 999/1000 \approx 0.4005$.

Código 2: Estrategia de w lineal decreciente

```
def w_lineal(wmax, wmin):
    # Enfoque 2: w = wmax - (wmax - wmin) * t / T
    def w(t, T, fitness, rng):
        return wmax - (wmax - wmin) * t / T
    return w
```

En esta estrategia w depende solo del número de iteración, por lo que todas las partículas usan el mismo valor en una misma iteración y la función devuelve un escalar.

3.2.2 Inercia basada en el fitness

La segunda estrategia calcula un w distinto para cada partícula en cada iteración:

$$w_i = \frac{1.1 \cdot \text{rand}()}{f(x_i)}, \tag{8}$$

donde $f(x_i)$ es el fitness de la posición actual de la partícula i y rand() es un número aleatorio uniforme en $[0, 1)$ generado de forma independiente para cada partícula. Como el problema es de minimización, una partícula con menor $f(x_i)$ recibe en promedio un w más grande y conserva más de su velocidad, mientras que las partículas con mal fitness pierden inercia y quedan dominadas por la atracción hacia p_i y g. El factor aleatorio evita que dos partículas con el mismo fitness reciban siempre el mismo valor.

Código 3: Función de peso por partícula

```
def w_fitness(usar_error=False, f_optimo=F_OPTIMO, epsilon=1e-12):
    # Enfoque 3: w_i = 1.1 * rand() / f(x_i), un valor por partícula
    def w(t, T, fitness, rng):
        f = fitness - f_optimo if usar_error else fitness
        return 1.1 * rng.random(fitness.shape) / np.maximum(f, epsilon)
    return w
```

El resultado de la ecuación (8) depende de la escala de la función objetivo. La función F5 del CEC 2017 incluye un desplazamiento de 500 en su valor, así que $f(x_i) \geq 500$ en todo el dominio y w_i nunca supera $1.1/500 = 0.0022$. Para estudiar este efecto se implementaron dos versiones controladas por el parámetro `usar_error`:

- Con `usar_error=False` se aplica la ecuación tal como está definida, usando el valor de $f(x_i)$ que devuelve el benchmark.
- Con `usar_error=True` el denominador es el error $f(x_i) - f^*$, con $f^* = 500$. De esta forma se elimina el desplazamiento de la función y w_i puede tomar

valores mayores. Cuando una partícula se acerca al óptimo el error tiende a cero y w_i crece sin límite, por lo que el denominador se acota inferiormente con `epsilon` para evitar una división entre cero.

3.2.3 Integración en el algoritmo

La función `ejecutar_pso` del cuaderno original calculaba w con la ecuación (7) dentro del ciclo. En la nueva versión recibe la estrategia como parámetro `estrategia_w` y la invoca en cada iteración con el fitness de las posiciones actuales. El fragmento siguiente muestra la parte del ciclo que cambió:

Código 4: Cálculo de w y actualización de velocidad dentro de `ejecutar_pso`

```
for it in range(num_iterations):
    r1 = rng.uniform(0, 1, size=(size, dimension))
    r2 = rng.uniform(0, 1, size=(size, dimension))

    w = np.asarray(estrategia_w(it, num_iterations, fitness, rng),
    ↪ dtype=float)
    historial_w[it] = w.mean()
    if w.ndim == 1:              # un w por particula
        w = w[:, None]

    velocity = (w * velocity +
                c1 * r1 * (personal_best_positions - position) +
                c2 * r2 * (global_best_position - position))

    if limitar:
        velocity = np.clip(velocity, -vmax, vmax)
        position = np.clip(position + velocity, low, high)
    else:
        position = position + velocity
```

El resultado de la estrategia se convierte a un arreglo de NumPy. Si es un escalar (estrategias constante y lineal), se multiplica directamente por la matriz de velocidades de tamaño $N \times D$. Si es un vector de N elementos (estrategia basada en fitness), se reordena como una columna de tamaño $N \times 1$ con `w[:, None]`, de modo que el *broadcasting* de NumPy multiplique toda la fila de velocidades de la partícula i por su propio w_i. En cada iteración también se guarda el promedio de w en `historial_w` para graficar su evolución.

Además, se agregó un límite a la velocidad, $v_{max} = 0.2\,(x_{max} - x_{min}) = 40$, y las posiciones se recortan al espacio de búsqueda $[-100, 100]^D$. Con $c_1 = c_2 = 1.5$, las configuraciones con w pequeño, como la basada en fitness, producen oscilaciones cada vez más amplias y las partículas salen del dominio. El límite se puede desactivar con `limitar=False`.

Por último, todas las configuraciones evaluadas se reúnen en un diccionario que asocia cada nombre con su enfoque y su estrategia. El experimento recorre este diccionario para cada dimensión y ejecuta 30 corridas por configuración:

Código 5: Configuraciones de w evaluadas

```
CONFIGURACIONES = {
    "Constante w=0.4":    ("Constante", w_constante(0.4)),
    "Constante w=0.6":    ("Constante", w_constante(0.6)),
    "Constante w=0.7298": ("Constante", w_constante(0.7298)),
    "Constante w=0.9":    ("Constante", w_constante(0.9)),
    "Lineal 0.9->0.4":    ("Lineal decreciente", w_lineal(0.9, 0.4)),
    "Fitness 1.1r/f(x)":  ("Basado en fitness",
    ↪ w_fitness(usar_error=False)),
    "Fitness 1.1r/error": ("Basado en fitness",
    ↪ w_fitness(usar_error=True)),
}
```

La ejecución k de cada configuración usa la semilla $2017 + 1000\,D + k$, la misma para todas las estrategias de w. Así, en una misma dimensión, todas las configuraciones parten de la misma población inicial y las diferencias en el desempeño se pueden atribuir a la estrategia de inercia. El desempeño de cada estrategia se analiza en la Sección 4.

4 Resultados

En esta sección se presentan los resultados de PSO sobre la función F5 (Shifted and Rotated Rastrigin) del benchmark CEC 2017 en 10, 20 y 30 dimensiones. Cada configuración del coeficiente de inercia w se ejecutó 30 veces con 50 partículas y 1000 iteraciones. La ejecución k de cada configuración partió de la misma población inicial, de modo que las diferencias entre configuraciones se deben a la estrategia de w y no a la inicialización. La métrica reportada es el error $f(x_{best}) - f^*$, donde $f^* = 500$ es el valor óptimo de F5. Ninguna ejecución alcanzó el óptimo global; el error más bajo registrado fue de 4.97, obtenido con w lineal decreciente en $D = 10$.

4.1 Error final por configuración

La Tabla 1 resume la media y la desviación estándar del error final. La mejor configuración de cada dimensión aparece en negritas. El símbolo entre paréntesis indica el resultado de la prueba de Wilcoxon pareada contra la mejor configuración, con corrección de Holm y $\alpha = 0.05$: (+) significa que la mejor configuración es significativamente superior y (=) que la diferencia no es significativa.

Cuadro 1: Error $f(x_{best}) - f^*$ de PSO en F5 (Rastrigin) (media ± desviación estándar, 30 ejecuciones).

Configuración de w	$D = 10$	$D = 20$	$D = 30$
Constante $w = 0.4$	29.442 ± 13.060 (+)	72.037 ± 19.543 (+)	185.764 ± 48.687 (+)
Constante $w = 0.6$	25.764 ± 8.035 (+)	64.029 ± 18.045 (+)	139.393 ± 41.407 (+)
Constante $w = 0.7298$	16.804 ± 5.990 (=)	55.078 ± 20.177 (+)	123.156 ± 33.203 (+)
Constante $w = 0.9$	26.465 ± 6.849 (+)	91.607 ± 16.288 (+)	222.272 ± 21.899 (+)
Lineal $0.9 \to 0.4$	**14.538 ± 5.690**	**37.764 ± 13.615**	**82.609 ± 29.148**
Fitness $1.1r/f(x)$	39.460 ± 12.969 (+)	101.298 ± 23.609 (+)	251.794 ± 40.230 (+)
Fitness $1.1r$/error	41.274 ± 12.024 (+)	104.239 ± 22.745 (+)	248.884 ± 38.504 (+)

El w lineal decreciente obtuvo el menor error medio en las tres dimensiones: 14.54 en $D = 10$, 37.76 en $D = 20$ y 82.61 en $D = 30$. La segunda posición correspondió siempre al w constante de 0.7298. En $D = 10$ la diferencia entre ambas no fue significativa ($p = 0.088$), pero en $D = 20$ y $D = 30$ sí lo fue, y la distancia entre ellas creció con la dimensión. En $D = 10$ el error de $w = 0.7298$ es 16 % mayor que el de la estrategia lineal, mientras que en $D = 20$ y $D = 30$ es 46 % y 49 % mayor, respectivamente.

El enfoque basado en fitness tuvo el peor desempeño en todas las dimensiones. Sus dos variantes, con $f(x)$ y con el error $f(x) - f^*$ en el denominador, dieron resultados casi idénticos (39.46 frente a 41.27 en $D = 10$ y 251.79 frente a 248.88 en $D = 30$). En $D = 30$ su error medio triplica el de la estrategia lineal.

La Figura 1 muestra los mismos datos en forma gráfica. La desviación estándar aumenta con la dimensión en todas las configuraciones. En $D = 30$ la más baja corresponde a $w = 0.9$ (21.90), aunque su media (222.27) es de las más altas.

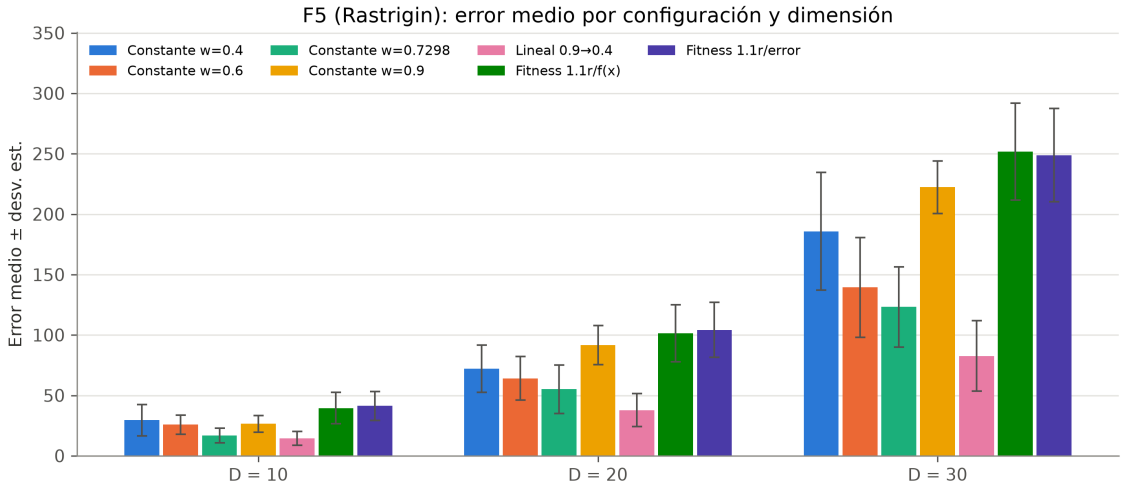


Figura 1: Error medio ± desviación estándar de cada configuración de w por dimensión.

4.2 Distribución del error

La Figura 2 muestra la distribución del error final de las 30 ejecuciones. En las tres dimensiones la caja de la estrategia lineal es la más baja, y en $D = 20$ y $D = 30$ su mediana (34.00 y 83.08) queda por debajo del primer cuartil de todas las demás configuraciones. En $D = 30$ su tercer cuartil prácticamente coincide con el primer cuartil de $w = 0.7298$.

Las dos variantes del enfoque basado en fitness tienen cajas similares entre sí y separadas del resto. En $D = 10$ son además las de mayor dispersión, con errores entre 15 y 72. El w constante de 0.4 también presenta una dispersión alta en las tres dimensiones y tiene la desviación estándar más grande de los valores constantes (13.06 en $D = 10$ y 48.69 en $D = 30$). Esto indica que su resultado depende mucho de la población inicial. Con $w = 0.9$ ocurre lo contrario en $D = 30$: la caja es estrecha pero está en un nivel de error alto, es decir, el algoritmo falla de manera consistente.

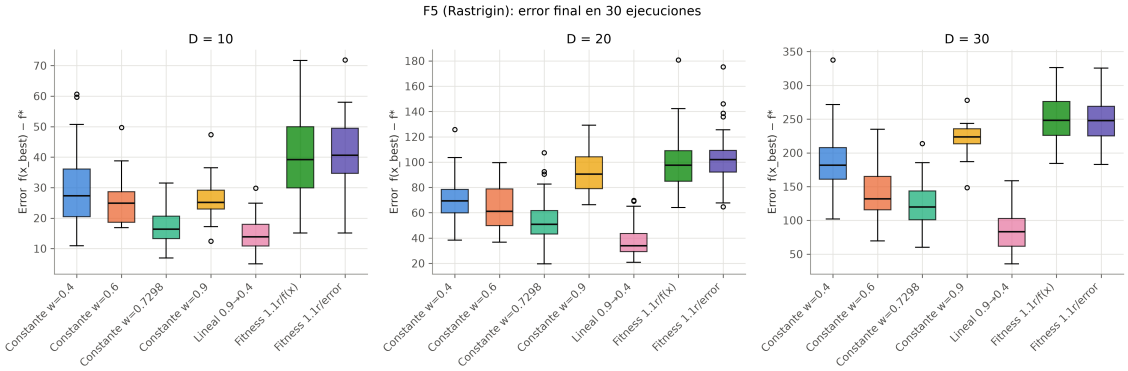


Figura 2: Distribución del error final en 30 ejecuciones para $D = 10$, 20 y 30.

4.3 Sensibilidad al valor de w y escalabilidad

La gráfica izquierda de la Figura 3 muestra el error medio en función del valor de w constante. En las tres dimensiones la relación tiene forma de U, con el mínimo en

$w = 0.7298$: valores menores provocan convergencia prematura y valores mayores impiden la convergencia. La forma se acentúa al aumentar la dimensión. En $D = 10$ el error medio varía entre 16.80 y 29.44 según el valor de w, y en $D = 30$ entre 123.16 y 222.27. Las líneas punteadas marcan el error de la estrategia lineal, que queda por debajo de todos los valores constantes probados en cada dimensión.

La gráfica derecha muestra cómo crece el error medio con la dimensión. Todas las configuraciones empeoran al pasar de 10 a 30 variables, y el crecimiento es más pronunciado entre $D = 20$ y $D = 30$ que entre $D = 10$ y $D = 20$. El orden de las configuraciones se mantiene casi igual en las tres dimensiones. La única excepción es $w = 0.9$, que en $D = 10$ tiene un error similar al de $w = 0.6$ (26.46 frente a 25.76) y en $D = 30$ es el peor de los valores constantes. La estrategia lineal pasa de 14.54 a 82.61, y es la configuración con menor error en todas las dimensiones.

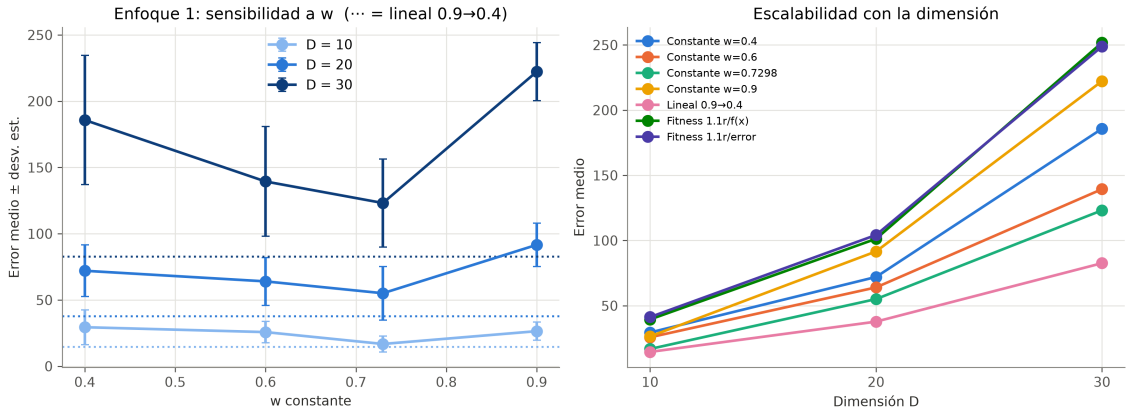


Figura 3: Izquierda: error medio en función de w constante (las líneas punteadas corresponden a la estrategia lineal $0.9 \rightarrow 0.4$). Derecha: error medio en función de la dimensión.

5 Resultados: coeficientes fijos frente a dinámicos

En esta sección se comparan dos implementaciones de PSO sobre la función F5 (Shifted and Rotated Rastrigin) del benchmark CEC 2017 en 10, 20 y 30 dimensiones. A diferencia de la Sección 4, en la que solo cambiaba el coeficiente de inercia w y se mantenía $c_1 = c_2 = 1.5$, aquí cambian los tres coeficientes w, c_1 y c_2. La primera implementación usa valores fijos durante toda la ejecución y la segunda los modifica en cada iteración (Tabla 2).

Cuadro 2: Coeficientes de las dos implementaciones de PSO.

Implementación	w	c_1 (cognitivo)	c_2 (social)
PSO fijo	0.7298	1.49618	1.49618
PSO dinámico	$0.9 \rightarrow 0.4$	$2.5 \rightarrow 0.5$	$0.5 \rightarrow 2.5$

Los valores fijos son los propuestos por Clerc y Kennedy [12]. En la implementación dinámica los tres coeficientes varían linealmente con la iteración t, siguiendo el esquema de coeficientes de aceleración variables en el tiempo (TVAC) de Ratnaweera et al. [16]:

$$w(t) = w_{max} - (w_{max} - w_{min})\,\frac{t}{T}, \tag{9}$$

$$c_1(t) = c_{1i} + (c_{1f} - c_{1i})\,\frac{t}{T}, \tag{10}$$

$$c_2(t) = c_{2i} + (c_{2f} - c_{2i})\,\frac{t}{T}, \tag{11}$$

donde $T = 1000$ es el número total de iteraciones. Al inicio domina el componente cognitivo, que favorece la exploración, y al final domina el social, que favorece la explotación alrededor del mejor global.

Cada implementación se ejecutó 30 veces por dimensión con 50 partículas y 1000 iteraciones. Se usaron las mismas semillas que en la Sección 4, de modo que la ejecución k de ambas implementaciones partió de la misma población inicial. La métrica reportada es el error $f(x_{best}) - f^*$, con $f^* = 500$.

5.1 Mejor solución, media y desviación estándar

La Tabla 3 presenta, para cada dimensión, la mejor solución encontrada en las 30 ejecuciones, junto con la media y la desviación estándar del error final. El mejor valor de cada dimensión aparece en negritas.

Cuadro 3: Resultados de PSO con coeficientes fijos y dinámicos en F5 (Rastrigin), 30 ejecuciones.

D	Implementación	Mejor $f(x)$	Mejor error	Media ± desv. est.	Peor error
10	PSO fijo	505.970	5.970	17.824 ± 6.720	33.415
	PSO dinámico	**503.980**	**3.980**	**8.884 ± 3.485**	**16.914**
20	PSO fijo	527.980	27.980	53.493 ± 17.778	99.493
	PSO dinámico	**511.940**	**11.940**	**24.343 ± 8.852**	**47.758**
30	PSO fijo	578.857	78.857	123.676 ± 28.868	217.894
	PSO dinámico	**541.902**	**41.902**	**68.677 ± 17.727**	**108.084**

La implementación dinámica obtuvo mejores resultados en todas las medidas y en las tres dimensiones. Su error medio es aproximadamente la mitad del de la implementación fija: 8.88 frente a 17.82 en $D = 10$, 24.34 frente a 53.49 en $D = 20$ y 68.68 frente a 123.68 en $D = 30$. Su desviación estándar también es menor (3.49, 8.85 y 17.73, frente a 6.72, 17.78 y 28.87), por lo que el resultado depende menos de la población inicial. Incluso la peor ejecución de la versión dinámica en $D = 20$ y $D = 30$ (47.76 y 108.08) queda por debajo del error medio de la versión fija.

La Figura 4 resume ambas implementaciones. A la izquierda se muestra el valor de cada coeficiente a lo largo de la ejecución y a la derecha el error final medio con su desviación estándar.

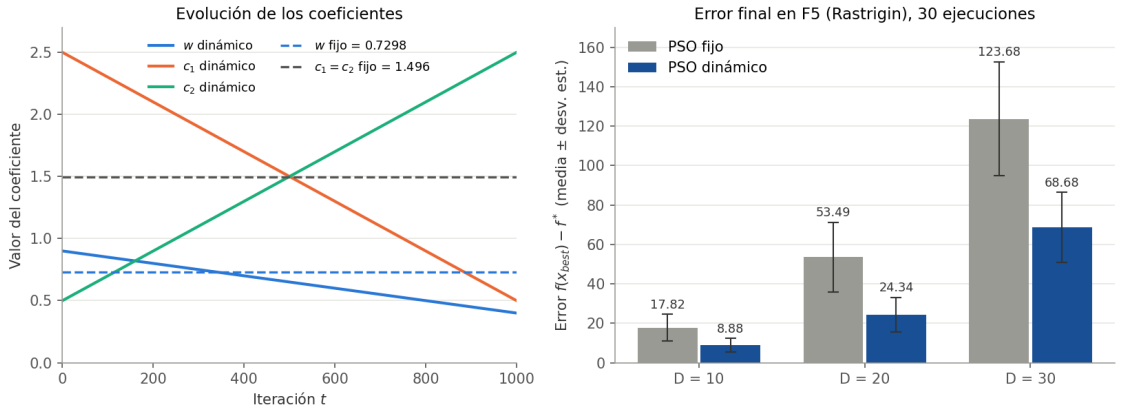


Figura 4: Izquierda: valor de w, c_1 y c_2 por iteración (líneas continuas: implementación dinámica; líneas punteadas: implementación fija, donde c_1 y c_2 coinciden). Derecha: error final medio ± desviación estándar en 30 ejecuciones para $D = 10$, 20 y 30.

Ninguna ejecución alcanzó el óptimo global. La mejor solución, con $f(x) = 503.98$, se obtuvo con coeficientes dinámicos en $D = 10$. Este error de 3.98 es menor que el mínimo registrado en la Sección 4 (4.97, con w lineal y $c_1 = c_2 = 1.5$).

5.2 Convergencia

La evolución del error del mejor global durante la ejecución, medida como la mediana de las 30 ejecuciones, explica la diferencia entre ambas implementaciones.

Con coeficientes fijos el error baja rápidamente y después se estanca. La mediana alcanza el 99 % de su mejora total alrededor de la iteración 143 en $D = 20$ y 186 en $D = 30$, y a partir de ahí la curva es prácticamente plana. En $D = 20$, por ejemplo, el error mediano es 49.5 tanto en la iteración 200 como en la 1000. El enjambre converge de forma prematura a un mínimo local de Rastrigin.

Con coeficientes dinámicos, las primeras iteraciones son más lentas. En la iteración 200 el error mediano es 27.2 en $D = 10$, 110.9 en $D = 20$ y 230.7 en $D = 30$, mayor que el de la versión fija en las tres dimensiones (19.9, 49.5 y 122.9). En esta etapa w y c_1 son altos y c_2 es bajo, así que cada partícula explora cerca de su propia trayectoria y la atracción hacia el mejor global es débil. Conforme c_2 aumenta y w disminuye, el error baja de forma pronunciada y la curva cruza a la de la versión fija cerca de la iteración 256 en $D = 10$, 301 en $D = 20$ y 376 en $D = 30$. El cruce ocurre más tarde al aumentar la dimensión. La versión dinámica sigue mejorando hasta la iteración 372, 392 y 456, respectivamente, antes de estabilizarse.

5.3 Pruebas estadísticas

Como las ejecuciones de ambas implementaciones comparten semilla, se aplicó la prueba de rangos con signo de Wilcoxon para muestras pareadas. Se eligió una prueba no paramétrica porque la prueba de Shapiro–Wilk rechazó la normalidad del error en $D = 20$ para ambas implementaciones y en $D = 30$ para la versión fija ($\alpha = 0.05$).

Cuadro 4: Prueba de Wilcoxon pareada entre PSO fijo y PSO dinámico.

	$D = 10$	$D = 20$	$D = 30$
Mediana del error (fijo)	16.42	49.53	117.90
Mediana del error (dinámico)	**8.89**	**22.88**	**65.67**
Ejecuciones en que gana el dinámico	27 / 30	29 / 30	29 / 30
Valor p	1.80×10^{-5}	5.59×10^{-9}	3.73×10^{-9}

La diferencia a favor de la implementación dinámica es significativa en las tres dimensiones ($p < 0.001$). Partiendo de la misma población inicial, la versión dinámica terminó con menor error en 27 de las 30 ejecuciones en $D = 10$ y en 29 de 30 en $D = 20$ y $D = 30$.

5.4 Comparación con los resultados de la Sección 4

Como se usaron las mismas semillas, los resultados también pueden compararse de forma pareada con las configuraciones de la Sección 4. La implementación fija no difiere significativamente de $w = 0.7298$ con $c_1 = c_2 = 1.5$ ($p = 0.349$, 0.584 y 0.855 en $D = 10$, 20 y 30), lo cual es esperable porque sus coeficientes son casi iguales.

La implementación dinámica mejora a la estrategia de w lineal $0.9 \to 0.4$ con $c_1 = c_2 = 1.5$, que había sido la mejor configuración. El error medio pasa de 14.54 a 8.88 en $D = 10$, de 37.76 a 24.34 en $D = 20$ y de 82.61 a 68.68 en $D = 30$. La diferencia es significativa en $D = 10$ y $D = 20$ ($p = 3.45 \times 10^{-5}$ en ambos casos), pero no en $D = 30$ ($p = 0.058$), donde la versión dinámica ganó en 21 de las 30 ejecuciones. Como ambas estrategias reducen w de la misma forma, esta mejora se

debe a la variación de c_1 y c_2: dar más peso al componente cognitivo al inicio y al social al final ayuda a evitar la convergencia prematura en Rastrigin.

6 Conclusiones

En esta práctica se implementó PSO en su versión clásica y se extendió con distintos esquemas de coeficientes dinámicos. Todas las configuraciones se evaluaron sobre la función F5 (Rastrigin) del CEC 2017 en 10, 20 y 30 dimensiones, con 30 ejecuciones por configuración y las mismas semillas, lo que permitió compararlas de forma pareada.

Cuando solo se modificó el peso de inercia, la estrategia lineal decreciente ($0.9 \rightarrow 0.4$) obtuvo el menor error medio en las tres dimensiones. Con w constante el error forma una U con el mínimo cerca de $w = 0.7298$: los valores bajos provocan convergencia prematura y los altos impiden converger. La estrategia basada en fitness tuvo el peor desempeño, aun después de eliminar el desplazamiento de la función.

Hacer variables también c_1 y c_2 (TVAC) dio los mejores resultados de toda la práctica. Frente a los coeficientes fijos, redujo el error medio aproximadamente a la mitad y también redujo su desviación estándar en las tres dimensiones, con diferencias significativas en todos los casos ($p < 0.001$). Las curvas de convergencia explican esta diferencia. Con coeficientes fijos el enjambre se estanca antes de la iteración 200, mientras que con coeficientes dinámicos avanza más lento al inicio pero sigue mejorando hasta pasadas las 350 iteraciones. Frente a la estrategia de w lineal con $c_1 = c_2 = 1.5$, la mejora fue significativa en $D = 10$ y $D = 20$ pero no en $D = 30$. Como ambas reducen w de la misma forma, esta ganancia se atribuye a dar más peso al componente cognitivo al inicio y al social al final.

En todas las configuraciones el error creció con la dimensión y ninguna ejecución alcanzó el óptimo global, lo que refleja la dificultad de Rastrigin por su gran cantidad de óptimos locales. El mejor resultado fue un error de 3.98 en $D = 10$, obtenido con coeficientes dinámicos.

Bibliografía

[1] Pereira, G. Particle Swarm Optimization. **2011**,

[2] Collet, P.; Rennard, J.-P. Stochastic Optimization Algorithms. 2007; `http://arxiv.org/abs/0704.3780`, arXiv:0704.3780 [cs.NE].

[3] Kennedy', J.; Eberhart, R. Particle Swarm Optimization. **1995**,

[4] Patel, P.; Marwala, T. Fuzzy Inference Systems Optimization. 2011; `http://arxiv.org/abs/1110.3385`, arXiv:1110.3385 [cs.AI].

[5] Freitas, D.; Lopes, L. G.; Morgado-Dias, F. Particle Swarm Optimisation: A Historical Review Up to the Current Developments. *Entropy* **2020**, *22*, 362.

[6] Nasr, M.; Farouk, O.; Mohamedeen, A.; Elrafie, A.; Bedeir, M.; Khaled, A. Benchmarking Meta-heuristic Optimization. *International Journal of Advanced Networking and Applications* **2020**, *11*, arXiv:2007.13476 [cs.NE].

[7] Poli, R.; Kennedy, J.; Blackwell, T. Particle swarm optimization: An overview. *Swarm Intelligence* **2007**, *1*, 33–57.

[8] Shi, Y.; Eberhart, R. C. Parameter selection in particle swarm optimization. Evolutionary Programming VII. Berlin, Heidelberg, 1998; pp 591–600.

[9] Shi, Y.; Eberhart, R. C. Empirical study of particle swarm optimization. Proceedings of the 1999 Congress on Evolutionary Computation (CEC99). Washington, DC, USA, 1999; pp 1945–1950.

[10] Shi, Y.; Eberhart, R. A modified particle swarm optimizer. Proceedings of the 1998 IEEE International Conference on Evolutionary Computation. Anchorage, AK, USA, 1998; pp 69–73.

[11] Kennedy, J.; Eberhart, R. Particle swarm optimization. Proceedings of ICNN'95 - International Conference on Neural Networks. Perth, WA, Australia, 1995; pp 1942–1948.

[12] Clerc, M.; Kennedy, J. The particle swarm - explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation* **2002**, *6*, 58–73.

[13] Eberhart, R. C.; Shi, Y. Comparing inertia weights and constriction factors in particle swarm optimization. Proceedings of the 2000 Congress on Evolutionary Computation. La Jolla, CA, USA, 2000; pp 84–88.

[14] Črepinšek, M.; Liu, S.-H.; Mernik, M. Exploration and exploitation in evolutionary algorithms: A survey. *ACM Computing Surveys* **2013**, *45*, 35:1–35:33.

[15] Zhan, Z.-H.; Zhang, J.; Li, Y.; Chung, H. S.-H. Adaptive particle swarm optimization. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* **2009**, *39*, 1362–1381.

[16] Ratnaweera, A.; Halgamuge, S. K.; Watson, H. C. Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients. *IEEE Transactions on Evolutionary Computation* **2004**, *8*, 240–255.

[17] Bansal, J. C.; Singh, P. K.; Saraswat, M.; Verma, A.; Jadon, S. S.; Abraham, A. Inertia weight strategies in particle swarm optimization. 2011 Third World Congress on Nature and Biologically Inspired Computing (NaBIC). Salamanca, Spain, 2011; pp 633–640.

[18] Chatterjee, A.; Siarry, P. Nonlinear inertia weight variation for dynamic adaptation in particle swarm optimization. *Computers & Operations Research* **2006**, *33*, 859–871.

[19] Eberhart, R. C.; Shi, Y. Tracking and optimizing dynamic systems with particle swarms. Proceedings of the 2001 Congress on Evolutionary Computation. Seoul, South Korea, 2001; pp 94–100.

[20] Nickabadi, A.; Ebadzadeh, M. M.; Safabakhsh, R. A novel particle swarm optimization algorithm with adaptive inertia weight. *Applied Soft Computing* **2011**, *11*, 3658–3670.